ORBIT · PYTHON · PAPER 08

第 8 套

60 题 · 完整题目与答案

单选 30 多选 15 填空 / 简答 15

本套分节

一、单项选择题（30） 二、不定项选择题（15） 三、填空题（5） 四、程序代码阅读题（5） 五、程序设计题（5）

答案与解析紧随每题；本文件用于复习核对，建议先独立作答。

一、单项选择题

01 • 单项选择

以下代码的输出是什么？

```
a = [1, 2]
b = a
a = a + [3]
print(b)
```

A. [1, 2, 3]
B. [1, 2]
C. [3]
D. 报错

答案 B. [1, 2]

解析 + vs += 的重要区别：

```
a = [1, 2]
b = a
# b和a指向同一个对象
a = a + [3]
# 创建新列表对象并赋值给a
print(b)
# [1, 2] - b仍指向原对象
```

原理：

- a + [3] 创建新列表对象
- a = ... 将a重新绑定到新对象
- b仍然指向原列表 [1, 2]

内存示意：

初始：a, b → [1, 2]
执行后：a → [1, 2, 3]（新对象）
b → [1, 2]（原对象）

02 • 单项选择

以下代码的输出是什么？

```
a = [1, 2]
b = a
a += [3]
print(b)
```

A. [1, 2, 3]
B. [1, 2]
C. [3]
D. 报错

答案 A. [1, 2, 3]

解析 += 就地修改列表：

```
a = [1, 2]
b = a
# b和a指向同一个对象
a += [3]
# 就地修改原列表
```

```
print(b)
# [1, 2, 3] - b也看到了修改
```

关键差异：

- += 对可变对象是就地操作
  就地操作（in-place）
- 不创建新对象，直接修改原对象
- 所有引用都能看到变化

对比总结：

```
# 对于列表（可变对象）
a = a + [3]
# 创建新对象
a += [3]
# 就地修改
# 对于元组（不可变对象）
t = t + (3,)
# 创建新对象
t += (3,)
# 也创建新对象（元组不可变）
```

03 • 单项选择

关于列表的is和==比较，以下说法正确的是：

```
a = [1, 2, 3]
b = [1, 2, 3]
```

A. a is b 为True

B. a == b 为True

C. 两者都为True

D. 两者都为False

答案 B. a == b 为True

解析 is vs == 的本质区别：

```
a = [1, 2, 3]
b = [1, 2, 3]
 == 比较值：
    比较值：
```

- 比较两个对象的内容
  内容是否相同
- a == b → True（内容相同）

```
 is 比较身份：
```

- 比较两个变量是否指向同一个对象
- a is b → False（不同对象）
- 等价于 id(a) == id(b)

示例对比：

```
a = [1, 2, 3]
b = [1, 2, 3]
# 新对象
c = a
# 同一对象
print(a == b)
# True - 内容相同
print(a is b)
```

```
# False - 不同对象
print(a is c)
# True - 同一对象
print(id(a), id(b), id(c))
```

04 • 单项选择

以下代码的输出是什么？

```
def modify(lst=[]):
    lst.append(len(lst))
    return lst
print(modify())
```

A. [0] [0] [0]
B. [0] [0, 1] [0, 1, 2]
C. [0] [1] [2]
D. 报错

答案 B. [0] [0, 1] [0, 1, 2]

解析 可变默认参数陷阱：

```
def modify(lst=[]):
    lst.append(len(lst))
    return lst
print(modify())
# [0]
print(modify())
                # [0, 1] - 共享同一个列表！
```

原因：

- 默认参数在函数定义时
  函数定义时创建，只创建一次
- 所有调用共享同一个默认参数对象
- 可变对象会累积修改

正确写法：

```
def modify(lst=None):
    if lst is None:
        lst = []
    # 每次创建新列表
    lst.append(len(lst))
    return lst
```

知识拓展：默认参数的创建时机：

```
def func(x, y=time.time()):
    # y在函数定义时求值
    return y
func(1)
# 每次返回相同时间戳
```

05 • 单项选择

关于元组的+操作，以下说法正确的是：

```
t1 = (1, 2)
t2 = t1 + (3,)
```

A. t1被修改

B. 创建新元组

C. 报错

D. t1和t2相同

答案 B. 创建新元组

解析 列表切片创建副本：

```
a = [1, 2, 3]
b = a[:]
# 切片创建副本
b.append(4)
print(a)
# [1, 2, 3] - 不受影响
```

切片的复制特性：

- a[:] 创建列表的浅拷贝
  浅拷贝
- 新列表，但元素是原对象的引用

其他复制方式：

```
import copy
# 浅拷贝
b = a.copy()
b = list(a)
b = a[:]
# 深拷贝（嵌套对象）
b = copy.deepcopy(a)
```

06 • 单项选择

以下代码的输出是什么？

```
t = (1, 2, 3)
t_id = id(t)
t = t + (4,)
print(id(t) == t_id)
```

A. True

B. False

C. 可能是True或False

D. 报错

答案 B. False

解析 字典的 get() 方法：

```
d = {'a': 1}
result = d.get('b', 0)
# 键不存在时返回默认值0
 get() vs 直接索引：
d = {'a': 1}
# 直接索引 - 键不存在会报错
value = d['b']
# KeyError
# get() - 键不存在返回默认值
value = d.get('b', 0)
# 返回0
value = d.get('b')
```

```
# 返回None
```

07 • 单项选择

关于字典的update()方法，以下说法正确的是：

A. 返回新字典
B. 修改原字典
C. 不修改任何字典
D. 创建副本

答案 B. 修改原字典

解析 集合的无序性：

```
s = {3, 1, 2}
print(s)
# 输出顺序不确定
```

集合特性：

- 无序
  无序：不保证元素顺序
- 唯一
  唯一：自动去重
- 可哈希元素
  可哈希元素：只能包含不可变对象

注意：Python 3.7+ 字典保持插入顺序，但集合仍然无序

08 • 单项选择

以下代码的输出是什么？

```
d1 = {'a': 1}
d2 = d1
d1.update({'b': 2})
print(len(d2))
```

A. 1
B. 2
C. 0
D. 报错

答案 B. 2

解析 浅拷贝与嵌套对象：

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a)
# 浅拷贝
b[0].append(3)
print(a)
# [[1, 2, 3], [3, 4]] - 内部列表被修改
```

浅拷贝 vs 深拷贝：

深拷贝：

```
# 浅拷贝：只复制第一层
b = copy.copy(a)
b = a.copy()
b = a[:]
# 深拷贝：递归复制所有层
```

```
b = copy.deepcopy(a)
```

09 • 单项选择

关于列表的remove()方法，以下说法正确的是：

A. 删除所有匹配的元素
B. 删除第一个匹配的元素
C. 返回删除的元素
D. 不修改原列表

答案 B. 删除第一个匹配的元素

解析 元组的不可变性是浅层的：

```
t = ([1, 2], 3)
t[0].append(3)
# 可以！修改列表内容
print(t)
# ([1, 2, 3], 3)
t[0] = [4, 5]
# 不可以！不能替换元素
```

原理：

- 元组本身不可变（不能修改元组结构）
- 但元组内的可变对象可以被修改

10 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3, 2, 1]
lst.remove(2)
print(lst)
```

A. [1, 3, 1]
B. [1, 3, 2, 1]
C. [1, 2, 3, 1]
D. [1, 1]

答案 B. [1, 3, 2, 1]

解析 字符串的不可变性：

```
s = "hello"
s[0] = 'H'
# TypeError: 字符串不支持修改
# 正确方式：创建新字符串
s = 'H' + s[1:]
# "Hello"
```

不可变对象修改的实质：

```
s = "hello"
s = s.upper()
# 创建新字符串"HELLO"
# 原字符串"hello"仍存在（如果有其他引用）
```

11 • 单项选择

关于集合的add()和update()，以下说法正确的是：

A. add()添加单个元素，update()添加多个元素

B. 两者完全相同

C. add()不修改原集合

D. update()返回新集合

答案 A. add()添加单个元素，update()添加多个元素

解析 字典键的要求：

```
d = {[1, 2]: 'value'}
                # TypeError: 列表不可哈希
```

可作为字典键的类型：

- 不可变类型：int, float, str, tuple, frozenset
- 必须是可哈希的（ hash(obj) 不报错）

示例：

```
# 正确
d = {(1, 2): 'a'}
# 元组可以
d = {'key': 'value'}
# 字符串可以
# 错误
d = {[1, 2]: 'a'}
# 列表不可以
d = {{1}: 'a'}
# 集合不可以
```

12 • 单项选择

以下代码的输出是什么？

```
s = {1, 2, 3}
s.update([3, 4, 5])
print(len(s))
```

A. 3

B. 5

C. 6

D. 8

答案 B. 5

解析 global 关键字的作用：

```
x = 10
def func():
    global x
    x = 20
# 修改全局变量
func()
print(x)
# 20
```

作用域规则（LEGB）：

```
                ）：
```

- Local：局部作用域
- Enclosing：闭包作用域
- Global：全局作用域
- Built-in：内置作用域

13 • 单项选择

关于字符串的replace()方法，以下说法正确的是：

A. 修改原字符串
B. 返回新字符串
C. 只替换第一个匹配
D. 不能使用

答案 B. 返回新字符串

解析 nonlocal 用于闭包：

```
def outer():
    x = 10
    def inner():
        nonlocal x
        x = 20
    # 修改外层函数的x
    inner()
    print(x)
# 20
 global vs nonlocal ：
x = 1
# 全局
def outer():
    x = 2
    # 外层
    def inner():
        nonlocal x
        # 引用外层的x
        x = 3
    inner()
    print(x)
# 3
print(x)
# 1 - 全局不变
```

14 • 单项选择

以下代码的输出是什么？

```
s = "hello"
s_id = id(s)
s = s.replace('l', 'L')
print(id(s) == s_id)
```

A. True
B. False
C. 可能是True或False
D. 报错

答案 B. False

解析 列表推导式中的变量作用域：

```
[x for x in range(3)]
print(x)
# Python 3: NameError
```

Python 2 vs 3 差异：
差异：
- Python 2: 列表推导式变量泄漏到外部
- Python 3: 列表推导式有自己的作用域

15 • 单项选择

关于列表的insert()方法，以下说法正确的是：

A. 返回新列表
B. 修改原列表
C. 不能在开头插入
D. 只能在末尾插入

答案 B. 修改原列表

解析 字典推导式：

```
{k: v for k, v in [('a', 1), ('b', 2)]}
# {'a': 1, 'b': 2}
```

推导式类型：

```
# 列表推导式
[x for x in range(3)]
# [0, 1, 2]
# 集合推导式
{x for x in range(3)}
# {0, 1, 2}
# 字典推导式
{x: x**2 for x in range(3)}
# {0:0, 1:1, 2:4}
# 生成器表达式
(x for x in range(3))
# generator object
```

16-30. 答案：

答案：A A B B B B B C B A B C C B A

这些题目涵盖了Python中可变性、数据结构操作、Pandas等关键概念。

16 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3]
lst.insert(0, 0)
print(lst)
```

A. [0, 1, 2, 3]
B. [1, 2, 3, 0]
C. [1, 0, 2, 3]
D. 报错

答案 A. [0, 1, 2, 3]

解析 （原卷合并作答）

17 • 单项选择

关于Pandas的at和iat，以下说法正确的是：

A. at使用标签，iat使用位置

B. 两者完全相同
C. at更快
D. iat返回副本

答案 A. at使用标签，iat使用位置

解析 （原卷合并作答）

18 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
df.at[0, 'A'] = 10
print(df.at[0, 'A'])
```

A. 1
B. 10
C. None
D. 报错

答案 A. 1

解析 （原卷合并作答）

19 • 单项选择

关于列表的count()方法，以下说法正确的是：

A. 修改列表
B. 返回元素出现次数
C. 删除元素
D. 返回列表长度

答案 A. 修改列表

解析 （原卷合并作答）

20 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 2, 3, 2]
print(lst.count(2))
```

A. 1
B. 2
C. 3
D. 5

答案 A. 1

解析 （原卷合并作答）

21 • 单项选择

关于字典的setdefault()方法，以下说法正确的是：

A. 不修改字典
B. 键存在时返回值，不存在时设置并返回默认值
C. 总是设置新值

D. 删除键

答案 A. 不修改字典

解析 （原卷合并作答）

22 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1}
x = d.setdefault('b', 2)
print(x, len(d))
```

A. None 1

B. 2 2

C. 1 1

D. 报错

答案 A. None 1

解析 （原卷合并作答）

23 • 单项选择

关于列表的index()方法，以下说法正确的是：

A. 修改列表

B. 返回元素第一次出现的索引

C. 元素不存在时返回-1

D. 返回所有索引

答案 A. 修改列表

解析 （原卷合并作答）

24 • 单项选择

以下代码会发生什么？

```
lst = [1, 2, 3]
idx = lst.index(4)
```

A. 返回-1

B. 返回None

C. 抛出ValueError

D. 返回3

答案 A. 返回-1

解析 （原卷合并作答）

25 • 单项选择

关于元组的count()和index()方法，以下说法正确的是：

A. 元组没有这些方法

B. 元组有这些方法且不修改元组

C. 这些方法会修改元组

D. 只有列表有这些方法

答案 A. 元组没有这些方法

解析 （原卷合并作答）

26 • 单项选择

以下代码的输出是什么？

```
t = (1, 2, 2, 3)
print(t.count(2), t.index(2))
```

A. 2 1
B. 2 2
C. 1 1
D. 报错

答案 A. 2 1

解析 （原卷合并作答）

27 • 单项选择

关于集合的discard()和remove()，以下说法正确的是：

A. 完全相同
B. discard()元素不存在时不报错，remove()会报错
C. discard()返回元素
D. remove()不修改集合

答案 A. 完全相同

解析 （原卷合并作答）

28 • 单项选择

以下代码会发生什么？

```
s = {1, 2, 3}
s.discard(4)
s.remove(5)
```

A. 正常执行
B. 第一行报错
C. 第二行报错
D. 都报错

答案 A. 正常执行

解析 （原卷合并作答）

29 • 单项选择

关于字典的pop()方法，以下说法正确的是：

A. 不修改字典
B. 删除并返回指定键的值
C. 只能删除最后一个键
D. 返回新字典

答案 A. 不修改字典

解析 （原卷合并作答）

30 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1, 'b': 2}
x = d.pop('a')
print(x, len(d))
```

A. 1 1
B. 1 2
C. None 1
D. 报错

答案 A. 1 1

解析 （原卷合并作答）

二、不定项选择题

31 • 不定项选择

以下哪些操作会改变列表的id？

A. `lst.append(1)`
B. `lst = lst + [1]`
C. `lst += [1]`
D. `lst.extend([1])`

答案 B. `lst = lst + [1]`

解析 关于 `+=` 操作符，只有B选项正确。 `+=` 对可变对象是就地修改，对不可变对象会创建新对象。

32 • 不定项选择

关于可变对象和不可变对象的性能，以下说法正确的有：

A. 不可变对象更安全
B. 可变对象修改更高效
C. 不可变对象适合作为字典键
D. 可变对象占用更多内存

答案 A. 不可变对象更安全；B. 可变对象修改更高效；C. 不可变对象适合作为字典键

解析 列表的可变性影响：
• A. 修改会影响所有引用
修改会影响所有引用　　所有引用指向同一对象
• B. 作为函数参数可能被修改
作为函数参数可能被修改　　传递引用
• C. 需要注意浅拷贝深拷贝
需要注意浅拷贝深拷贝　　嵌套结构问题
• D. 不能作为字典的值
不能作为字典的值　　可以作为字典的值

33 • 不定项选择

以下哪些方法返回新对象而不修改原对象？

A. `str.upper()`
B. `list.sort()`

C. tuple + tuple

D. dict.get()

答案 A. str.upper()；C. tuple + tuple；D. dict.get()

解析 元组的特性：

• A. 元组是不可变的

元组是不可变的 元组本身不可变

• B. 元组内的元素都不可变

元组内的元素都不可变 元组可包含可变对象

• C. 元组可以包含列表

元组可以包含列表 可以包含任何对象

• D. 元组可以作为字典键

元组可以作为字典键 元组是可哈希的

示例：

```
t = ([1, 2], 3)
# 元组包含列表
t[0].append(3)
# 可以修改列表内容
d = {t: 'value'} # TypeError - 包含列表的元组不可哈希
```

34 • 不定项选择

关于列表的切片操作，以下说法正确的有：

A. lst[:]创建浅拷贝

B. lst[1:3]返回新列表

C. 切片赋值会修改原列表

D. 切片不能用于元组

答案 A. lst[:]创建浅拷贝；B. lst[1:3]返回新列表；C. 切片赋值会修改原列表

解析 字典的可变性：

• A. 字典是可变的

字典是可变的 可以添加删除键值对

• B. 字典的键必须是不可变的

字典的键必须是不可变的 键必须可哈希

• C. 字典的值可以是任何类型

字典的值可以是任何类型 没有限制

• D. 字典保证有序

字典保证有序 Python 3.7+保持插入顺序，但这是实现细节

35 • 不定项选择

以下哪些是字典的视图对象？

A. keys()

B. values()

C. items()

D. get()

答案

解析

36 • 不定项选择

关于函数参数的默认值，以下说法正确的有：

A. 默认值在函数定义时计算
B. 可变默认值会被共享
C. 每次调用都会重新计算默认值
D. 应避免使用可变对象作为默认值

答案

解析

37 • 不定项选择

以下哪些操作会创建新的字符串对象？

A. `s.upper()`
B. `s + "hello"`
C. `s * 2`
D. `s[0]`

答案

解析

38 • 不定项选择

关于Pandas的copy参数，以下说法正确的有：

A. copy=True创建副本
B. copy=False可能返回视图
C. 默认值通常是True
D. 影响性能

答案

解析

39 • 不定项选择

以下哪些类型是可哈希的？

A. `int`
B. `str`
C. `tuple`
D. `list`

答案

解析

40 • 不定项选择

关于列表的`extend()`和`append()`，以下说法正确的有：

A. `extend()`添加多个元素
B. `append()`添加单个元素（作为整体）
C. 两者都修改原列表
D. `extend()`更快

答案

解析

41 • 不定项选择

以下哪些操作不会抛出异常？

A. `[1, 2, 3].index(4)`
B. `{1, 2, 3}.discard(4)`
C. `{'a': 1}.get('b')`
D. `(1, 2, 3)[5]`

答案

解析

42 • 不定项选择

关于集合的运算，以下说法正确的有：

A. `s1 | s2` 返回并集
B. `s1 & s2` 返回交集
C. `s1 - s2` 返回差集
D. 这些运算不修改原集合

答案

解析

43 • 不定项选择

以下哪些方法会修改原字典？

A. `update()`
B. `pop()`
C. `setdefault()`
D. `get()`

答案

解析

44 • 不定项选择

关于元组的特性，以下说法正确的有：

A. 元组比列表快
B. 元组占用更少内存
C. 元组可以作为字典键
D. 元组完全不可变

答案

解析

45 • 不定项选择

以下哪些是避免修改外部对象的方法？

A. 传递副本
B. 在函数内创建新对象
C. 使用不可变对象
D. 使用global关键字

答案

解析

三、填空题

46 • 填空 / 简答

列表的 _____ 操作符会修改原列表，而 _____ 操作符会创建新列表。

答案 `+=` ， 就地修改

解析 就地修改 （或
（或 修改原对象
修改原对象 ）
`+=` 对可变对象和不可变对象的行为不同。

```
# 可变对象 - 就地修改
lst = [1, 2]
id_before = id(lst)
lst += [3]
print(id(lst) == id_before)
# True - 同ID
# 不可变对象 - 创建新对象
tup = (1, 2)
id_before = id(tup)
tup += (3,)
print(id(tup) == id_before)
                # False - 不ID
```

47 • 填空 / 简答

字典的 _____ 方法会修改原字典，而 _____ 方法只是查询不修改。

答案 `update()` ， `setdefault()`

解析 字典的两个常用修改方法。

```
# update - 批量更新
dict1 = {'a': 1, 'b': 2}
dict1.update({'b': 3, 'c': 4})
print(dict1)
# {'a': 1, 'b': 3, 'c': 4}
# setdefault - 设置默认值
dict2 = {'a': 1}
val = dict2.setdefault('b', 2)
# 返回2
print(dict2)
# {'a': 1, 'b': 2}
val = dict2.setdefault('a', 999)
# 返回1
print(dict2)
# {'a': 1, 'b': 2} - a未变
```

48 • 填空 / 简答

在函数中，如果要修改全局变量，需要使用 _____ 关键字；如果要修改外层函数的变量，需要使用 _____ 关键字。

答案 global ， nonlocal

解析 修改外部变量的关键字。

```
# global - 修改全局变量
x = 10
def modify_global():
    global x
    x = 20
modify_global()
print(x)
# 20
# nonlocal - 修改外层函数变量
def outer():
    x = 10
    def inner():
        nonlocal x
        x = 20
    inner()
    print(x)
# 20
outer()
```

49 • 填空 / 简答

Pandas中，_____ 会给出SettingWithCopyWarning警告，应该使用 _____ 或 _____ 来避免。

答案 is ， ==

解析 身份比较与值比较。

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b)
# True - 值相等
print(a is b)
# False - 不同对象
print(a is c)
# True - 同一对象
# 小整数缓存
x = 256
y = 256
print(x is y)
# True - 缓存
x = 257
y = 257
print(x is y)
# False - 超出缓存范围
```

50 • 填空 / 简答

集合的 _____ 方法删除元素时如果元素不存在会报错，而 _____ 方法不会报错。

答案 copy() ， view

解析 Pandas的副本和视图概念。

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
# 明确创建副本
df_copy = df.copy()
df_copy.loc[0, 'A'] = 999
print(df)
# 原数据未变
# 视图（可能）
df_view = df[['A']]
# 可能是视图
# 注意：Pandas 2.0+使用COW机制
```

1. 答案：

答案： `+=` ， `+`

两种运算符的差异：

`+=` 就地修改：

```
a = [1, 2]
b = a
a += [3]
# 就地修改，a和b都变为[1,2,3]
```

`+` 创建新对象：

```
a = [1, 2]
b = a
a = a + [3]
# 创建新对象，a=[1,2,3]，b=[1,2]
```

关键点： `+=` 对可变对象是in-place操作， `+` 总是创建新对象。

2. 答案：

答案： `update()` （或

（或 `pop()` 等），

等）， `get()`

字典的修改方法vs查询方法：

修改原字典的方法：

- `update()` ：更新多个键值对
- `pop()` ：删除并返回值
- `popitem()` ：删除并返回键值对
- `setdefault()` ：设置默认值
- `clear()` ：清空字典

不修改原字典的方法：

- `get()` ：获取值（不存在返回默认值）
- `keys()` , `values()` , `items()` ：返回视图对象
- `copy()` ：返回浅拷贝

对比示例：

```
d = {'a': 1}
# 修改方法
d.update({'b': 2})
# d变为{'a':1, 'b':2}
d.pop('a')
# 删除'a'，返回1
# 查询方法
value = d.get('c', 0)
# 返回0，d不变
```

```
keys = d.keys()
# 返回视图，d不变
```

3. 答案：

答案： global ， nonlocal

作用域关键字：

global - 声明全局变量：

```
x = 10
# 全局变量
def func():
    global x
    # 声明使用全局x
    x = 20
# 修改全局变量
func()
print(x)
# 20
```

nonlocal - 声明外层变量：

```
def outer():
    x = 10
    # 外层变量
    def inner():
        nonlocal x
        # 声明使用外层x
        x = 20
    # 修改外层变量
    inner()
    print(x)
# 20
outer()
```

使用场景：

- global ：在函数内修改全局变量
- nonlocal ：在嵌套函数中修改外层函数的变量（闭包）

注意：如果只是读取变量，不需要声明；只有修改时才需要。

4. 答案：

答案： 链式索引

链式索引 ， loc ， iloc

Pandas中的安全索引方法：

链式索引的问题：

```
df[df['A'] > 1]['B'] = 100
# 可能在副本上修改，产生警告
```

正确的索引方法：

loc - 基于标签：

```
df.loc[df['A'] > 1, 'B'] = 100
# 正确
df.loc['row_label', 'col_label']
# 使用标签
```

iloc - 基于位置：

```
df.iloc[0:5, 2:4]
# 使用整数位置
```

为什么要避免链式索引：

- 可能在临时副本上操作

- 行为不确定（可能修改原数据，也可能不修改）
- Pandas会发出 SettingWithCopyWarning

最佳实践：

```
# 错误
df[df['A'] > 1]['B'] = 100
# 正确
df.loc[df['A'] > 1, 'B'] = 100
```

5. 答案：

答案： remove() ， discard()

集合删除元素的两种方法：

remove() - 删除元素，不存在则报错：

```
s = {1, 2, 3}
s.remove(2)
# 成功，s变为{1, 3}
s.remove(5)
# KeyError: 5
```

discard() - 删除元素，不存在不报错：

```
s = {1, 2, 3}
s.discard(2)
# 成功，s变为{1, 3}
s.discard(5)
# 不报错，s不变
```

使用建议：

- 确定元素存在时用 remove()
- 不确定元素是否存在时用 discard()

其他删除方法：

```
s.pop()
# 删除并返回任意元素
s.clear()
# 清空集合
```

对比总结：

```
s = {1, 2, 3}
# remove: 严格模式
try:
    s.remove(5)
except KeyError:
    print("元素不存在")
# discard: 宽松模式
s.discard(5)
# 静默忽略
```

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
def mystery_function(n, cache={}):
    if n in cache:
        return cache[n]
    if n <= 1:
```

```
        result = n
    else:
        result = mystery_function(n-1) + mystery_function(n-2)
    cache[n] = result
    return result
print(mystery_function(5))
print(mystery_function(10))
print(len(mystery_function.__defaults__[0]))
```

问题：

- (1) 这个函数计算什么？
- (2) 使用可变默认参数的优缺点是什么？
- (3) 最后一个print输出什么？

答案 - (1) 这个函数计算斐波那契数列
- (2) 优点：利用可变默认参数实现简单的缓存机制，避免重复计算。缺点：缓存是全局的，多次调用会累积，可能导致内
- (3) 输出：11（缓存中有11个键值对：0-10的斐波那契数）

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
class ShoppingCart:
    items = []
    def add_item(self, item):
        self.items.append(item)
    def get_items(self):
        return self.items
cart1 = ShoppingCart()
cart2 = ShoppingCart()
cart1.add_item("Apple")
cart2.add_item("Banana")
print(cart1.get_items())
print(cart2.get_items())
```

问题：

- (1) 两个print的输出是什么？
- (2) 这是什么问题？
- (3) 如何修复？

答案 - (1) 输出：
['Apple', 'Banana']
- (2) items是类属性而非实例属性，所有实例共享同一个列表
- (3) 修复方法：
```python
class ShoppingCart:
    def __init__(self):
        self.items = [] # 在__init__中创建实例属性
```

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
# 操作1
df['C'] = df['A'] + df['B']
# 操作2
df.loc[:, 'D'] = df['A'] * 2
# 操作3
subset = df[df['A'] > 1]
subset.loc[:, 'E'] = 100
print('E' in df.columns)
```

问题：

- (1) 输出是什么？
- (2) 为什么操作3不会修改df？
- (3) 如何修改操作3使其能修改df？

答案 • (1) 输出：False（或可能产生警告）

- (2) subset是df的一个副本（或视图），对subset的修改不会影响df
- (3) 修改方法：使用loc直接修改： df.loc[df['A'] > 1, 'E'] = 100

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
def process_data(data, transform=lambda x: x * 2):
    result = []
    for item in data:
        result.append(transform(item))
    return result
numbers = [1, 2, 3, 4, 5]
doubled = process_data(numbers)
tripled = process_data(numbers, lambda x: x * 3)
print(numbers)
print(doubled)
print(tripled)
```

问题：

- (1) 三个print的输出分别是什么？
- (2) 为什么numbers没有被修改？
- (3) 这种设计模式的优点是什么？

答案 • (1) 输出：

```
[1, 2, 3, 4, 5]
[2, 4, 6, 8, 10]
[3, 6, 9, 12, 15]
```

- (2) process_data创建新列表，不修改原列表
- (3) 优点：无副作用、可测试性强、可复用性好、易于理解

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
def create_multipliers(n):
    multipliers = []
    for i in range(n):
        multipliers.append(lambda x: x * i)
    return multipliers
funcs = create_multipliers(3)
print([f(10) for f in funcs])
def create_multipliers_fixed(n):
    multipliers = []
    for i in range(n):
        multipliers.append(lambda x, i=i: x * i)
    return multipliers
funcs_fixed = create_multipliers_fixed(3)
print([f(10) for f in funcs_fixed])
```

问题：

- (1) 两个print的输出分别是什么？
- (2) 第一个函数有什么问题？
- (3) 第二个函数如何解决这个问题？

答案 • (1) 输出：

```
[20, 20, 20]
[0, 10, 20]
```

- (2) 第一个函数中所有lambda都引用同一个变量i，循环结束后i=2
- (3) 第二个函数使用默认参数 i=i 在定义时捕获i的值

解析

五、程序设计题

56 • 填空 / 简答

实现一个工具来追踪对象的可变性变化：

1. MutabilityTracker类：
    - 包装任何对象
    - 记录所有修改操作
    - 提供修改历史查询
2. 功能要求

功能要求：

- 拦截所有修改方法
- 记录修改前后的状态
- 支持撤销操作
- 导出修改日志

3. 支持的类型

支持的类型：

- 列表
- 字典

- 集合
- 自定义对象

答案

```
import copy
from functools import wraps
from datetime import datetime
class MutabilityTracker:
    """
    可变性追踪器 - 追踪对象的所有修改操作
    功能特性：
    1. 记录所有修改操作
    2. 保存修改前后的状态
    3. 支持撤销操作
    4. 导出修改日志
    使用示例：
    tracker = MutabilityTracker([1, 2, 3])
    tracker.append(4)
    tracker.undo()
    """
    def __init__(self, obj):
        """
        初始化追踪器
        参数：
        - obj: 要追踪的对象（列表、字典或集合）
        """
        self._obj = obj
        self._history = []
        self._operation_log = []
        # 保存初始状态
        self._save_state("初始化", copy.deepcopy(obj))
    def _save_state(self, operation, state_before=None):
        """保存操作前的状态"""
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        if state_before is None:
            state_before = copy.deepcopy(self._obj)
    self._history.append({
        'timestamp': timestamp,
        'operation': operation,
        'state_before': state_before,
        'state_after': copy.deepcopy(self._obj)
    })
    self._operation_log.append(f"[{timestamp}] {operation}")
def __getattr__(self, name):
    """拦截所有方法调用"""
    attr = getattr(self._obj, name)
    # 如果是修改方法，包装它
    if callable(attr) and self._is_mutating_method(name):
        @wraps(attr)
```

```
        def wrapper(*args, **kwargs):
            # 保存操作前状态
            state_before = copy.deepcopy(self._obj)
            # 执行操作
            result = attr(*args, **kwargs)
            # 记录操作
            args_str = ', '.join(repr(a) for a in args)
            self._save_state(f"{name}({args_str})", state_before)
            return result
        return wrapper
    return attr
def _is_mutating_method(self, method_name):
    """判断是否是修改方法"""
    mutating_methods = {
        # 列表方法
        'append', 'extend', 'insert', 'remove', 'pop',
        'clear', 'sort', 'reverse',
        # 字典方法
        'update', 'setdefault', 'popitem',
        # 集合方法
        'add', 'discard', 'update', 'intersection_update',
        'difference_update', 'symmetric_difference_update'
    }
    return method_name in mutating_methods
def undo(self):
    """撤销最后一次操作"""
    if len(self._history) <= 1:
        print("没有可撤销的操作")
        return
    # 获取上一个状态
    self._history.pop()
    # 移除当前状态
    previous_state = self._history[-1]['state_after']
    # 恢复状态
    if isinstance(self._obj, list):
        self._obj.clear()
        self._obj.extend(previous_state)
    elif isinstance(self._obj, dict):
        self._obj.clear()
        self._obj.update(previous_state)
    elif isinstance(self._obj, set):
        self._obj.clear()
        self._obj.update(previous_state)
    print(f"已撤销到：{previous_state}")
def get_history(self):
    """获取修改历史"""
    return self._history.copy()
def export_log(self, filename=None):
```

```
    """导出修改日志"""
    log = "\n".join(self._operation_log)
    if filename:
        with open(filename, 'w', encoding='utf-8') as f:
            f.write(log)
            print(f"日志已导出到{filename}")
        else:
            print("=== 修改日志 ===")
            print(log)
        return log
    def get_obj(self):
        """获取被追踪的对象"""
        return self._obj
    def __repr__(self):
        return f"MutabilityTracker({self._obj})"
# === 使用示例 === {#-使用示例- }
print("=== 可变性追踪器示例 ===\n")
# 示例1：追踪列表 {#示例1追踪列表 }
print("示例1：追踪列表操作")
tracker = MutabilityTracker([1, 2, 3])
print(f"初始状态: {tracker.get_obj()}")
tracker.append(4)
print(f"添加后: {tracker.get_obj()}")
tracker.extend([5, 6])
print(f"扩展后: {tracker.get_obj()}")
tracker.remove(2)
print(f"删除后: {tracker.get_obj()}")
# 查看历史 {#查看历史 }
print("\n修改历史:")
for record in tracker.get_history()[-3:]:
    print(f"
            {record['operation']}: {record['state_before']} -> {record['state_after']}")
# 撤销操作 {#撤销操作 }
print("\n撤销最后一次操作:")
tracker.undo()
print(f"当前状态: {tracker.get_obj()}")
# 导出日志 {#导出日志 }
print()
tracker.export_log()
# 示例2：追踪字典 {#示例2追踪字典 }
print("\n\n示例2：追踪字典操作")
dict_tracker = MutabilityTracker({'a': 1, 'b': 2})
print(f"初始: {dict_tracker.get_obj()}")
dict_tracker.update({'c': 3, 'd': 4})
print(f"更新后: {dict_tracker.get_obj()}")
dict_tracker.pop('b')
print(f"删除后: {dict_tracker.get_obj()}")
dict_tracker.undo()
```

```
print(f"撤销后: {dict_tracker.get_obj()}")
```

解析

57 • 填空 / 简答

实现自动处理函数参数可变性的工具：

1. 装饰器

装饰器@safe_params：

- 自动检测可变参数
- 创建参数副本
- 可配置深拷贝/浅拷贝

2. 装饰器

装饰器@immutable_return：

- 将返回值转为不可变
- 支持嵌套结构
- 提供转换报告

3. 工具函数

工具函数：

- make_immutable(obj)
- is_deeply_immutable(obj)
- compare_mutability(obj1, obj2)

答案

```
import copy
from functools import wraps
import inspect
def safe_params(deep=False):
    """
    自动安全化参数装饰器
    功能：
    1. 自动检测可变参数
    2. 创建参数副本
    3. 可配置深拷贝/浅拷贝
    使用示例：
    @safe_params(deep=True)
    def my_function(data):
        data.append(1) # 不会影响外部
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            # 处理位置参数
            safe_args = []
            for arg in args:
                if _is_mutable(arg):
                safe_args.append(copy.deepcopy(arg) if deep else copy.copy(arg))
                else:
                safe_args.append(arg)
            # 处理关键字参数
            safe_kwargs = {}
```

```
            for key, value in kwargs.items():
                if _is_mutable(value):
                safe_kwargs[key] = copy.deepcopy(value) if deep else copy.copy(value)
                else:
                safe_kwargs[key] = value
            return func(*safe_args, **safe_kwargs)
        return wrapper
    return decorator
def immutable_return(func):
    """
    将返回值转为不可变装饰器
    功能：
    1. list -> tuple
    2. dict -> frozenset of items
    3. set -> frozenset
    4. 支持嵌套结构
    """
    @wraps(func)
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return _make_immutable(result)
    return wrapper
def _is_mutable(obj):
    """判断对象是否可变"""
    immutable_types = (int, float, str, tuple, frozenset, bytes, bool, type(None))
    return not isinstance(obj, immutable_types)
def _make_immutable(obj):
    """递归转换为不可变对象"""
    if isinstance(obj, list):
        return tuple(_make_immutable(item) for item in obj)
    elif isinstance(obj, dict):
        return frozenset((k, _make_immutable(v)) for k, v in obj.items())
    elif isinstance(obj, set):
        return frozenset(_make_immutable(item) for item in obj)
    else:
        return obj
def is_deeply_immutable(obj, visited=None):
    """
    检查对象是否深度不可变
    参数：
    - obj: 要检查的对象
    - visited: 已访问的对象集合（处理循环引用）
    返回：
    - bool: 是否深度不可变
    """
    if visited is None:
        visited = set()
    # 避免循环引用
```

```
    obj_id = id(obj)
    if obj_id in visited:
        return True
    visited.add(obj_id)
    # 检查基本不可变类型
    if isinstance(obj, (int, float, str, bytes, bool, type(None))):
        return True
    # 检查frozenset
    if isinstance(obj, frozenset):
        return all(is_deeply_immutable(item, visited) for item in obj)
    # 检查元组
    if isinstance(obj, tuple):
        return all(is_deeply_immutable(item, visited) for item in obj)
    # 其他都是可变的
    return False
def compare_mutability(obj1, obj2):
    """
    比较两个对象的可变性
    返回：
    - dict: 包含两个对象的可变性信息
    """
    return {
        'obj1': {
            'is_mutable': _is_mutable(obj1),
            'is_deeply_immutable': is_deeply_immutable(obj1),
            'type': type(obj1).__name__
        },
        'obj2': {
            'is_mutable': _is_mutable(obj2),
            'is_deeply_immutable': is_deeply_immutable(obj2),
            'type': type(obj2).__name__
        }
# === 使用示例 === {#-使用示例--1 }
print("=== 函数参数安全化工具示例 ===\n")
# 示例1：安全参数装饰器 {#示例1安全参数装饰器 }
@safe_params(deep=True)
def modify_data(data):
    """这个函数会修改数据，但不影响外部"""
    data.append(999)
    if isinstance(data, list) and len(data) > 1:
        data[0] = {'modified': True}
    return data
original = [{'value': 1}, {'value': 2}]
print(f"原始数据: {original}")
result = modify_data(original)
print(f"返回值: {result}")
print(f"原始数据未变: {original}\n")
# 示例2：不可变返回值 {#示例2不可变返回值 }
```

```
@immutable_return
def create_data():
    """返回可变数据，但会被转换为不可变"""
    return {
        'items': [1, 2, 3],
        'settings': {'debug': True},
        'tags': {'python', 'programming'}
    }
data = create_data()
print(f"返回的数据（已冻结）: {data}")
print(f"数据类型: {type(data)}\n")
# 示例3：深度不可变性检查 {#示例3深度不可变性检查 }
test_cases = [
    (1, 2, 3),
    ([1, 2], 3),
    ((1, 2), (3, 4)),
    frozenset([1, 2, 3])
]
print("深度不可变性检查:")
for obj in test_cases:
    is_immutable = is_deeply_immutable(obj)
    print(f"
            {obj}: {is_immutable}")
# 示例4：比较可变性 {#示例4比较可变性 }
print("\n可变性比较:")
comparison = compare_mutability([1, 2, 3], (1, 2, 3))
for obj_name, info in comparison.items():
    print(f"
            {obj_name}: {info}")
```

解析

58 • 填空 / 简答

实现不可变配置对象：

1. ImmutableConfig类：

- 初始化后不可修改
- 支持嵌套配置
- 提供只读访问

2. ConfigBuilder类：

- 构建器模式
- 验证配置
- 生成不可变配置

3. 功能

功能：

- 从文件加载
- 配置合并
- 配置导出

答案

```
from collections.abc import Mapping
import json
from pathlib import Path
class ImmutableConfig(Mapping):
    """
    不可变配置对象
    特性：
    1. 初始化后完全不可修改
    2. 支持嵌套配置访问
    3. 提供只读的字典接口
    4. 可序列化
    使用示例：
    config = ImmutableConfig({'database': {'host': 'localhost'}})
    print(config['database']['host'])
    """
    def __init__(self, data):
        """
        初始化配置
        参数：
        - data: 配置字典
        """
        # 递归转换嵌套字典为ImmutableConfig
        object.__setattr__(self, '_data', self._make_immutable(data))
        object.__setattr__(self, '_hash', None)
    def _make_immutable(self, obj):
        """递归转换为不可变配置"""
        if isinstance(obj, dict):
            return {k: self._make_immutable(v) for k, v in obj.items()}
        elif isinstance(obj, list):
            return tuple(self._make_immutable(item) for item in obj)
        elif isinstance(obj, set):
            return frozenset(self._make_immutable(item) for item in obj)
    else:
        return obj
# 实现Mapping接口
def __getitem__(self, key):
    """获取配置项"""
    value = self._data[key]
    if isinstance(value, dict):
        return ImmutableConfig(value)
    return value
def __iter__(self):
    """迭代键"""
    return iter(self._data)
def __len__(self):
    """配置项数量"""
    return len(self._data)
# 防止修改
```

```
def __setitem__(self, key, value):
    """禁止设置"""
    raise TypeError("不可修改不可变配置")
def __delitem__(self, key):
    """禁止删除"""
    raise TypeError("不可修改不可变配置")
def __setattr__(self, name, value):
    """禁止设置属性"""
    raise AttributeError("不可修改不可变配置的属性")
# 哈希支持
def __hash__(self):
    """使配置可哈希"""
    if self._hash is None:
        object.__setattr__(self, '_hash', hash(self._to_hashable(self._data)))
    return self._hash
def _to_hashable(self, obj):
        """转换为可哈希对象"""
        if isinstance(obj, dict):
            return tuple(sorted((k, self._to_hashable(v)) for k, v in obj.items()))
        elif isinstance(obj, (list, tuple)):
            return tuple(self._to_hashable(item) for item in obj)
        elif isinstance(obj, set):
            return frozenset(self._to_hashable(item) for item in obj)
        else:
            return obj
    def __repr__(self):
        """字符串表示"""
        return f"ImmutableConfig({self._data})"
    def to_dict(self):
        """转换为普通字典"""
        return self._data.copy()
class ConfigBuilder:
    """
    配置构建器
    功能：
    1. 使用构建器模式创建配置
    2. 支持链式调用
    3. 配置验证
    4. 生成不可变配置
    使用示例：
    config = (ConfigBuilder()
        .set('database', {'host': 'localhost'})
        .set('port', 8000)
        .validate()
        .build())
    """
    def __init__(self):
        """初始化构建器"""
```

```
        self._config = {}
    self._validators = []
def set(self, key, value):
    """设置配置项"""
    keys = key.split('.')
    current = self._config
    for k in keys[:-1]:
        if k not in current:
            current[k] = {}
        current = current[k]
    current[keys[-1]] = value
    return self
def merge(self, other_config):
    """合并其他配置"""
    self._deep_merge(self._config, other_config)
    return self
def _deep_merge(self, base, update):
    """深度合并字典"""
    for key, value in update.items():
        if key in base and isinstance(base[key], dict) and isinstance(value, dict):
            self._deep_merge(base[key], value)
        else:
            base[key] = value
def add_validator(self, validator_func):
    """添加验证器"""
    self._validators.append(validator_func)
    return self
def validate(self):
    """验证配置"""
    for validator in self._validators:
        if not validator(self._config):
            raise ValueError(f"配置验证失败: {validator.__name__}")
    return self
    def load_from_file(self, filepath):
        """从文件加载配置"""
        path = Path(filepath)
        if path.suffix == '.json':
            with path.open('r', encoding='utf-8') as f:
                config = json.load(f)
            self.merge(config)
        return self
    def build(self):
        """构建不可变配置"""
        return ImmutableConfig(self._config)
# === 使用示例 === {#-使用示例--2 }
print("=== 不可变配置系统示例 ===\n")
# 示例1：创建不可变配置 {#示例1创建不可变配置 }
config_data = {
```

```
    'database': {
        'host': 'localhost',
        'port': 5432,
        'credentials': {
            'user': 'admin',
            'password': 'secret'
    },
    'app': {
        'debug': False,
        'workers': 4
    },
    'features': ['auth', 'api', 'cache']
}
config = ImmutableConfig(config_data)
print(f"配置: {config}")
print(f"数据库主机: {config['database']['host']}")
print(f"功能列表: {config['features']}")
# 尝试修改（会失败） {#尝试修改会失败 }
try:
    config['database']['host'] = 'newhost'
except TypeError as e:
    print(f"\n   修改被阻止: {e}")
# 示例2：使用构建器 {#示例2使用构建器 }
print("\n使用ConfigBuilder:")
# 定义验证器 {#定义验证器 }
def validate_port(config):
    """验证端口号"""
    if 'database' in config and 'port' in config['database']:
        port = config['database']['port']
        return 1024 <= port <= 65535
    return True
# 构建配置 {#构建配置 }
built_config = (ConfigBuilder()
    .set('database.host', 'localhost')
    .set('database.port', 5432)
    .set('app.debug', True)
    .add_validator(validate_port)
    .validate()
    .build())
print(f"构建的配置: {built_config}")
print(f"配置哈希值: {hash(built_config)}")
# 配置可以作为字典键 {#配置可以作为字典键 }
config_cache = {built_config: "cached_data"}
print(f"\n配置作为字典键: {len(config_cache)} 个条目")
```

评分说明：

- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分

- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析

59 • 填空 / 简答

实现数据对象的快照功能：

1. Snapshot类：
- 保存对象状态
- 支持恢复
- 最小化存储

2. SnapshotManager：
- 管理多个快照
- 自动快照
- 快照比较

3. 应用

应用：
- 数据库事务模拟
- 游戏存档
- 配置回滚

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析

60 • 填空 / 简答

实现Pandas操作的安全检查工具：

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析